

Complete Flask Industrial Yield Optimization Dashboard

This document contains the remaining core implementation files for the professional Flask dashboard.

FILE 1 — app.py

```
from flask import Flask, render_template
import pandas as pd
import plotly.express as px
import plotly
import json

app = Flask(__name__)

# Load Dataset

df = pd.read_csv('data/manufacturing_data.csv')

@app.route('/')
def dashboard():

    avg_yield = round(df['yield_percentage'].mean(), 2)
    avg_scrap = round(df['scrap_rate'].mean(), 2)
    avg_quality = round(df['quality_score'].mean(), 2)
    avg_energy = round(df['energy_consumption'].mean(), 2)

    # Yield Trend
    trend_df = (
        df.groupby('timestamp')['yield_percentage']
        .mean()
        .reset_index()
    )

    line_chart = px.line(
        trend_df,
        x='timestamp',
        y='yield_percentage',
        title='Yield Trend Over Time'
    )

    line_graphJSON = json.dumps(
```

```

        line_chart,
        cls=plotly.utils.PlotlyJSONEncoder
    )

    # Plant Benchmarking
    benchmark_df = (
        df.groupby('plant_id')[
            ['yield_percentage', 'scrap_rate']
        ]
        .mean()
        .reset_index()
    )

    bar_chart = px.bar(
        benchmark_df,
        x='plant_id',
        y='yield_percentage',
        color='scrap_rate',
        title='Cross Plant Benchmarking'
    )

    bar_graphJSON = json.dumps(
        bar_chart,
        cls=plotly.utils.PlotlyJSONEncoder
    )

    return render_template(
        'dashboard.html',
        avg_yield=avg_yield,
        avg_scrap=avg_scrap,
        avg_quality=avg_quality,
        avg_energy=avg_energy,
        line_graphJSON=line_graphJSON,
        bar_graphJSON=bar_graphJSON
    )

@app.route('/monitoring')
def monitoring():

    sensor_df = df.tail(300)

    sensor_chart = px.line(
        sensor_df,
        x='timestamp',
        y=['temperature', 'pressure', 'humidity'],
        title='Process Sensor Monitoring'
    )

```

```

sensor_graphJSON = json.dumps(
    sensor_chart,
    cls=plotly.utils.PlotlyJSONEncoder
)

return render_template(
    'monitoring.html',
    sensor_graphJSON=sensor_graphJSON
)

@app.route('/benchmarking')
def benchmarking():

    benchmark_df = (
        df.groupby('plant_id')[
            [
                'yield_percentage',
                'scrap_rate',
                'quality_score',
                'energy_consumption'
            ]
        ]
        .mean()
        .reset_index()
    )

    return render_template(
        'benchmarking.html',
        tables=[benchmark_df.to_html(classes='table table-dark table-striped')]
    )

@app.route('/recommendations')
def recommendations():

    latest = df.tail(12).to_dict(orient='records')

    recommendations_data = []

    for row in latest:

        recommendation = []

        if row['humidity'] > 60:
            recommendation.append('Reduce humidity below 55%')

```

```

        if row['vibration'] > 5:
            recommendation.append('Inspect machine vibration levels')

        if row['temperature'] < 175:
            recommendation.append('Increase process temperature by 3°C')

        if len(recommendation) == 0:
            recommendation.append('Process operating optimally')

        row['recommendations'] = recommendation

        recommendations_data.append(row)

    return render_template(
        'recommendations.html',
        data=recommendations_data
    )

@app.route('/semantic-layer')
def semantic_layer():

    semantic_kpis = [
        {
            'name': 'Yield Percentage',
            'definition': 'Good Units / Total Units',
            'owner': 'Manufacturing',
            'source': 'Historian'
        },
        {
            'name': 'Scrap Rate',
            'definition': 'Scrap Units / Total Production',
            'owner': 'Production',
            'source': 'MES'
        },
        {
            'name': 'Quality Score',
            'definition': 'Weighted Quality Metric',
            'owner': 'Quality Team',
            'source': 'LIMS'
        },
        {
            'name': 'Energy Consumption',
            'definition': 'Energy Usage Per Batch',
            'owner': 'Operations',
            'source': 'IoT Sensors'
        }
    ]

```

```
    return render_template(
        'semantic_layer.html',
        kpis=semantic_kpis
    )

if __name__ == '__main__':
    app.run(debug=True)
```

FILE 2 — templates/base.html

```
<!DOCTYPE html>
<html lang="en">
<head>

    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Industrial Yield Optimization</title>

    <link
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
bootstrap.min.css"
        rel="stylesheet"
    >

    <link
        rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/font-awesome@6.5.2/css/
all.min.css"
    >

    <link
        rel="stylesheet"
        href="{{ url_for('static', filename='css/style.css') }}"
    >

    <script src="https://cdn.plot.ly/plotly-latest.min.js"></script>

</head>
<body>

<div class="sidebar">
```

```

<div>
  <h2 class="logo"><img alt="Smart Factory AI logo" data-bbox="365 128 382 142"/> Smart Factory AI</h2>
</div>

<ul class="nav-links">
  <li><a href="/"><i class="fas fa-chart-line"></i> Dashboard</a></li>
  <li><a href="/monitoring"><i class="fas fa-microchip"></i> Monitoring</a></li>
  <li><a href="/benchmarking"><i class="fas fa-industry"></i> Benchmarking</a></li>
  <li><a href="/recommendations"><i class="fas fa-robot"></i> AI Recommendations</a></li>
  <li><a href="/semantic-layer"><i class="fas fa-database"></i> Semantic Layer</a></li>
</ul>

</div>

<div class="main-content">

  {% block content %}
  {% endblock %}

</div>

</body>
</html>

```

FILE 3 — templates/dashboard.html

```

{% extends 'base.html' %}

{% block content %}

<h1 class="page-title">Executive Dashboard</h1>

<div class="row g-4 mb-4">

  <div class="col-md-3">
    <div class="kpi-card">
      <h5>Average Yield</h5>
      <h1>{{ avg_yield }}%</h1>
    </div>

```

```

</div>

<div class="col-md-3">
  <div class="kpi-card">
    <h5>Scrap Rate</h5>
    <h1>{{ avg_scrap }}%</h1>
  </div>
</div>

<div class="col-md-3">
  <div class="kpi-card">
    <h5>Quality Score</h5>
    <h1>{{ avg_quality }}</h1>
  </div>
</div>

<div class="col-md-3">
  <div class="kpi-card">
    <h5>Energy Consumption</h5>
    <h1>{{ avg_energy }}</h1>
  </div>
</div>

</div>

<div class="chart-card mb-5">
  <div id="linechart"></div>
</div>

<div class="chart-card">
  <div id="barchart"></div>
</div>

<script>

  var lineGraphs = {{ line_graphJSON | safe }};
  Plotly.newPlot('linechart', lineGraphs.data, lineGraphs.layout);

  var barGraphs = {{ bar_graphJSON | safe }};
  Plotly.newPlot('barchart', barGraphs.data, barGraphs.layout);

</script>

{% endblock %}

```

FILE 4 — templates/monitoring.html

```
{% extends 'base.html' %}

{% block content %}

<h1 class="page-title">Process Monitoring</h1>

<div class="chart-card">
  <div id="sensorchart"></div>
</div>

<script>

  var sensorGraphs = {{ sensor_graphJSON | safe }};
  Plotly.newPlot(
    'sensorchart',
    sensorGraphs.data,
    sensorGraphs.layout
  );

</script>

{% endblock %}
```

FILE 5 — templates/benchmarking.html

```
{% extends 'base.html' %}

{% block content %}

<h1 class="page-title">Cross Plant Benchmarking</h1>

<div class="table-card">

  {% for table in tables %}
    {{ table|safe }}
  {% endfor %}

</div>
```



```
{% endblock %}
```

FILE 6 — templates/recommendations.html

```
{% extends 'base.html' %}

{% block content %}

<h1 class="page-title">AI Recommendation Center</h1>

<div class="row">

{% for row in data %}

<div class="col-md-6 mb-4">

    <div class="recommendation-card">

        <h4>{{ row.plant_id }}</h4>

        <p><strong>Predicted Yield:</strong> {{ row.predicted_yield }}%</p>

        <p><strong>Current Yield:</strong> {{ row.yield_percentage }}%</p>

        <h5>Recommendations</h5>

        <ul>
            {% for rec in row.recommendations %}
                <li>{{ rec }}</li>
            {% endfor %}
        </ul>

    </div>

</div>

{% endfor %}

</div>

{% endblock %}
```

FILE 7 — templates/semantic_layer.html

```
{% extends 'base.html' %}

{% block content %}

<h1 class="page-title">Semantic KPI Governance Layer</h1>

<div class="table-card">

<table class="table table-dark table-striped">

    <thead>
        <tr>
            <th>KPI</th>
            <th>Definition</th>
            <th>Owner</th>
            <th>Source</th>
        </tr>
    </thead>

    <tbody>

        {% for kpi in kpis %}

            <tr>
                <td>{{ kpi.name }}</td>
                <td>{{ kpi.definition }}</td>
                <td>{{ kpi.owner }}</td>
                <td>{{ kpi.source }}</td>
            </tr>

        {% endfor %}

    </tbody>

</table>

</div>

{% endblock %}
```

FILE 8 — static/css/style.css

```
body {
  margin: 0;
  background-color: #0B132B;
  color: white;
  font-family: Arial, sans-serif;
}

.sidebar {
  position: fixed;
  width: 260px;
  height: 100vh;
  background-color: #1C2541;
  padding: 25px;
}

.logo {
  color: #00D9FF;
  margin-bottom: 40px;
}

.nav-links {
  list-style: none;
  padding: 0;
}

.nav-links li {
  margin-bottom: 20px;
}

.nav-links a {
  text-decoration: none;
  color: white;
  font-size: 18px;
  transition: 0.3s;
}

.nav-links a:hover {
  color: #00D9FF;
}

.main-content {
  margin-left: 280px;
  padding: 30px;
}
```

```
.page-title {  
  margin-bottom: 30px;  
  color: #00D9FF;  
}  
  
.kpi-card {  
  background-color: #1C2541;  
  padding: 25px;  
  border-radius: 15px;  
  text-align: center;  
  box-shadow: 0 4px 15px rgba(0,0,0,0.3);  
  transition: 0.3s;  
}  
  
.kpi-card:hover {  
  transform: translateY(-5px);  
}  
  
.kpi-card h1 {  
  color: #00D9FF;  
}  
  
.chart-card {  
  background-color: #1C2541;  
  padding: 25px;  
  border-radius: 15px;  
  margin-bottom: 30px;  
}  
  
.table-card {  
  background-color: #1C2541;  
  padding: 25px;  
  border-radius: 15px;  
}  
  
.recommendation-card {  
  background-color: #1C2541;  
  padding: 25px;  
  border-radius: 15px;  
  box-shadow: 0 4px 15px rgba(0,0,0,0.3);  
}  
  
.recommendation-card h4 {  
  color: #00D9FF;  
}
```

FILE 9 — requirements.txt

```
flask
pandas
numpy
plotly
faker
scikit-learn
joblib
unicorn
```

FINAL PROJECT STRUCTURE

```
yield-optimization-platform/
|
├─ app.py
├─ requirements.txt
|
├─ data/
│   └─ manufacturing_data.csv
|
├─ simulator/
│   └─ generate_data.py
|
├─ static/
│   └─ css/
│       └─ style.css
|
└─ templates/
    ├─ base.html
    ├─ dashboard.html
    ├─ monitoring.html
    ├─ benchmarking.html
    ├─ recommendations.html
    └─ semantic_layer.html
```

HOW TO RUN

STEP 1

Install dependencies:

```
pip install -r requirements.txt
```

STEP 2

Generate dataset:

```
python simulator/generate_data.py
```

STEP 3

Run Flask app:

```
python app.py
```

WHAT THIS VERSION ALREADY HAS

- ✓ Enterprise dashboard UI
- ✓ Sidebar navigation
- ✓ KPI cards
- ✓ Plotly interactive charts
- ✓ Process monitoring
- ✓ Benchmarking
- ✓ AI recommendations

✓ Semantic layer governance page

✓ Professional industrial theme

✓ Multi-page architecture

NEXT LEVEL ENTERPRISE UPGRADES

Now we transform the project into a premium industrial AI platform.

PHASE 2 — ADVANCED ENTERPRISE FEATURES

New Features We Will Add

1. Real ML Yield Prediction

Upgrade from static recommendations to actual ML predictions.

Add:

- RandomForestRegressor
 - Yield prediction API
 - Scrap prediction
 - Confidence score
 - Feature importance
-

FILE — models/train_model.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
import joblib

# Load Data

df = pd.read_csv('data/manufacturing_data.csv')

features = [
    'temperature',
```

```

        'pressure',
        'humidity',
        'machine_speed',
        'vibration',
        'raw_material_quality',
        'energy_consumption',
        'downtime_minutes'
    ]

    X = df[features]
    y = df['yield_percentage']

    # Split
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )

    # Model
    model = RandomForestRegressor(
        n_estimators=200,
        max_depth=12,
        random_state=42
    )

    model.fit(X_train, y_train)

    # Predictions
    preds = model.predict(X_test)

    mae = mean_absolute_error(y_test, preds)

    print(f'Model MAE: {mae}')

    # Save Model
    joblib.dump(model, 'models/yield_model.pkl')

    print('Model saved successfully')

```


FILE — models/predict.py

```
import joblib
import pandas as pd

model = joblib.load('models/yield_model.pkl')

def predict_yield(data):

    input_df = pd.DataFrame([data])

    prediction = model.predict(input_df)[0]

    return round(prediction, 2)
```

PHASE 3 — ADVANCED DASHBOARD UI

Add Gauge Charts

Install:

```
pip install dash-bootstrap-components
```

Add Gauge Chart in Dashboard

app.py

Add:

```
import plotly.graph_objects as go
```

Create Gauge

```
gauge_chart = go.Figure(go.Indicator(  
    mode='gauge+number',  
    value=avg_yield,  
    title={'text': 'Yield Efficiency'},  
    gauge={  
        'axis': {'range': [0, 100]},  
        'bar': {'color': '#00D9FF'},  
        'steps': [  
            {'range': [0, 70], 'color': '#ff4d4d'},  
            {'range': [70, 85], 'color': '#ffaa00'},  
            {'range': [85, 100], 'color': '#00ff99'}  
        ]  
    }  
))
```

PHASE 4 — REAL-TIME SENSOR SIMULATION

Add Auto Refresh

Install:

```
pip install flask-socketio
```

FILE — app.py

```
from flask_socketio import SocketIO  
  
socketio = SocketIO(app)
```

Add Live Data Emission

```
@socketio.on('connect')
def connect():
    print('Client connected')
```

PHASE 5 — DIGITAL TWIN UI LOOK

Add Industrial Status Panels

Add cards for:

- Machine Health
- Process Stability
- Sensor Status
- Production Efficiency
- Downtime Alerts

Example HTML Card

```
<div class="status-card healthy">
  <h5>Machine Health</h5>
  <h1>Healthy</h1>
</div>
```

Add CSS

```
.status-card {
  padding: 25px;
  border-radius: 15px;
  text-align: center;
  margin-bottom: 20px;
}

.healthy {
  background: linear-gradient(135deg, #00b894, #00cec9);
```

```
}

.warning {
    background: linear-gradient(135deg, #fdbc6e, #e17055);
}

.critical {
    background: linear-gradient(135deg, #d63031, #e84393);
}
```

PHASE 6 — AI INSIGHTS ENGINE

Create Automated AI Insights

FILE — `utils/insights_engine.py`

```
import pandas as pd

def generate_insights(df):
    insights = []

    high_humidity = df[df['humidity'] > 60]

    if len(high_humidity) > 0:
        insights.append(
            'High humidity strongly impacts yield reduction.'
        )

    high_vibration = df[df['vibration'] > 5]

    if len(high_vibration) > 0:
        insights.append(
            'Machine vibration increases scrap probability.'
        )

    low_quality = df[df['quality_score'] < 80]

    if len(low_quality) > 0:
        insights.append(
            'Low quality batches detected in Plant B.'
        )
```

```
return insights
```

PHASE 7 — LOGIN SYSTEM

Add Authentication

Install:

```
pip install flask-login
```

Add Roles

Roles:

- Admin
- Plant Manager
- Process Engineer
- Operator

PHASE 8 — SQL DATABASE

Upgrade From CSV

Use:

- SQLite initially
- PostgreSQL later

FILE — database/schema.sql

```
CREATE TABLE manufacturing_data (  
    id INTEGER PRIMARY KEY,  
    timestamp DATETIME,  
    plant_id TEXT,
```

```
production_line TEXT,  
temperature FLOAT,  
pressure FLOAT,  
humidity FLOAT,  
machine_speed FLOAT,  
vibration FLOAT,  
yield_percentage FLOAT,  
scrap_rate FLOAT,  
quality_score FLOAT  
);
```

PHASE 9 — ADVANCED CHARTS

Add:

Heatmaps

```
import plotly.express as px  
  
corr = df.corr(numeric_only=True)  
  
fig = px.imshow(  
    corr,  
    text_auto=True,  
    title='Correlation Heatmap'  
)
```

Radar Charts

Compare plants.

Sankey Diagrams

Show production flow.

PHASE 10 — EXPORT REPORTS

Add PDF Download

Install:

```
pip install reportlab
```

Export KPI Reports

Generate:

- Yield reports
 - Plant reports
 - AI recommendation reports
-

PHASE 11 — AI CHATBOT ASSISTANT

Add Manufacturing Assistant

Examples:

```
User:
Why is Plant B yield decreasing?

AI:
High humidity and vibration spikes are correlated with yield reduction.
```

PHASE 12 — FASTAPI MICROSERVICES

Recommended Architecture Upgrade

Later convert:

```
Flask Frontend
  ↓
FastAPI ML Service
  ↓
Prediction API
```

Example FastAPI Prediction Service

prediction_api.py

```
from fastapi import FastAPI
import joblib
import pandas as pd

app = FastAPI()

model = joblib.load('models/yield_model.pkl')

@app.post('/predict')
def predict(data: dict):

    input_df = pd.DataFrame([data])

    prediction = model.predict(input_df)[0]

    return {
        'predicted_yield': round(prediction, 2)
    }
```

PHASE 13 — INDUSTRIAL UI IMPROVEMENTS

Add:

- Glassmorphism cards
- Animated gradients
- Neon industrial theme
- Hover animations
- Live blinking alerts
- Sidebar animations
- Fullscreen dashboard

- Responsive mobile layout

ADVANCED CSS EXAMPLE

```
.kpi-card {  
  background: rgba(255,255,255,0.08);  
  backdrop-filter: blur(10px);  
  border: 1px solid rgba(255,255,255,0.1);  
}  
  
.kpi-card:hover {  
  transform: translateY(-8px);  
  transition: 0.3s ease;  
}
```

PHASE 14 — BUSINESS STORYTELLING FEATURES

Add Executive Sections

Business Impact

Show:

- Yield increase
- Cost savings
- Scrap reduction
- Energy savings
- CO₂ reduction

Example KPI Impact Cards

Estimated Annual Savings
\$2.4M

Scrap Reduction
14%

Yield Improvement
8.2%

FINAL ENTERPRISE ARCHITECTURE

Frontend Layer

- |
- ├─ Flask UI
- ├─ Bootstrap Dashboard
- ├─ Plotly Visualizations
- └─ JavaScript Animations

Backend Layer

- |
- ├─ Flask APIs
- ├─ FastAPI Prediction Services
- ├─ Recommendation Engine
- └─ AI Insights Engine

Data Layer

- |
- ├─ Historian Data
- ├─ Sensor Streams
- ├─ LIMS Data
- ├─ SQL Database
- └─ Semantic KPI Layer

ML Layer

- |
- ├─ Yield Prediction
- ├─ Scrap Prediction
- ├─ Anomaly Detection
- └─ Recommendation Models

NEXT BEST STEP

Now build these immediately in order:

1. train_model.py
2. predict.py
3. Gauge charts
4. AI insights page
5. Correlation heatmap
6. Live industrial alerts

7. Login system
8. SQL database
9. FastAPI prediction service
10. Executive business impact dashboard

This will transform the project from a dashboard demo into a production-style industrial AI platform.